

《操作系统习题》重点题目答案

目录

1	操作系统概述	1
2	进程与线程	2
3	CPU 调度	6
4	进程同步与死锁	9
5	内存管理	13
6	文件系统	19

1 操作系统概述

题目 1 并发与并行

Compare the concepts of concurrency and parallelism.

答案

Concurrency is the interleaved progression of multiple tasks over a period of time; parallelism is the simultaneous execution of multiple tasks at the exact same moment. Parallelism must involve running at the same time, whereas concurrency does not necessarily require it.

题目 2 双模式执行

What is dual-mode of execution in OS? What are its benefits?

答案

Dual-mode execution means the processor provides at least two execution modes: user mode and kernel mode. User programs typically run in user mode, while switching to kernel mode occurs when executing privileged instructions, accessing critical hardware resources, or handling interrupts and exceptions. The benefits of this approach include achieving protection, security, stability, and controlled resource access.

题目 3 系统调用

What is the purpose of system calls?

答案

The role of system calls is to provide user programs with a standard interface for entering the operating system kernel and requesting OS services, such as process control, file operations, device access, memory management, and communication.

2 进程与线程

题目 4 使用 `fork()` 生成斐波那契数列

The Fibonacci sequence... Write a C program using the `fork()` system call that generates the Fibonacci sequence in the child process.

答案

Below is a program that directly implements the requirements: the parent process creates a child process, the child process outputs the first n terms of the Fibonacci sequence, and the parent process waits for the child process to terminate.

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>

int main(int argc, char *argv[]) {
    if (argc != 2) {
        fprintf(stderr, "Usage: %s <n>\n", argv[0]);
        return 1;
    }

    int n = atoi(argv[1]);
    if (n <= 0) {
        fprintf(stderr, "n must be positive\n");
        return 1;
    }

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    } else if (pid == 0) {
        int a = 0, b = 1;
        for (int i = 0; i < n; ++i) {
            if (i == 0) {
                printf("%d", a);
            } else if (i == 1) {
                printf(" %d", b);
            } else {
                int c = a + b;
                printf(" %d", c);
                a = b;
                b = c;
            }
        }
        printf("\n");
        return 0;
    } else {
        wait(NULL);
        return 0;
    }
}
```

题目 5 fork() 输出分析

要求如下：已知从这个程序执行到这个程序的所有进程结束这个时间段内，没有其它新进程执行。

```
#include "stdio.h"
#include "sys/types.h"
#include "unistd.h"

int main() {
    pid_t pid1;
    pid_t pid2;
    pid1 = fork();
    pid2 = fork();
    printf("pid1:%d, pid2:%d\n", pid1, pid2);
}
```

1. 请说出执行这个程序后，将一共运行几个进程。
2. 如果其中一个进程的输出结果是“pid1:1001, pid2:1002”，写出其他进程的输出结果（不考虑进程执行顺序），并加以解释。

答案

1. 一共运行 4 个进程。
2. 其余 3 个进程的输出是：
 - a. pid1:0, pid2:1003
 - b. pid1:1001, pid2:0
 - c. pid1:0, pid2:0

题目 6 程序将创建多少个进程

- (a) In the following program, how many processes will be created?
- (b) (optional)

```
#include <unistd.h>
#include <stdio.h>

int main() {
    pid_t pid;
    int a = 5, b = 7;
    pid = fork();

    if (pid < 0) {
        fprintf(stderr, "Fork Failed");
        exit(-1);
    } else if (pid == 0) {
        a = a * 2;
        printf("this is Child process, a=%d, b=%d\n", a, b);
    } else {
        printf("this is parent process, a=%d, b=%d\n", a, b);
        wait(NULL);
        a = a + 1;
        b = b + 3;
        printf("Child Complete, a=%d, b=%d\n", a, b);
        exit(0);
    }
}
```

```
#include <unistd.h>
#include <stdio.h>

int main() {
    fork();
    fork() && fork() || fork();
    fork();
    sleep(100);
    return 0;
}
```

答案

- 1. 第一个程序最终共有 2 个进程，其中新创建 1 个子进程。
- 2. 第一个程序的输出内容为：
 - a. 子进程: `this is Child process, a=10, b=7`
 - b. 父进程第一次输出: `this is parent process, a=5, b=7`
 - c. 父进程等待子进程结束后输出: `Child Complete, a=6, b=10`
- 3. 第二个程序最终共有 20 个进程，其中新创建 19 个子进程。

3 CPU 调度

题目 7 短程、中程、长程调度

Describe the differences among short-term, medium-term, and long-term scheduling.

答案

Short-term scheduling is responsible for selecting processes from the ready queue to occupy the CPU; medium-term scheduling is responsible for suspending and swapping processes in and out of the memory; long-term scheduling is responsible for deciding which jobs enter the system.

题目 8 五进程调度综合题

Consider the following set of processes, with the length of the CPU-burst time given in milliseconds:

Process	Burst Time	Priority
P1	10	3
P2	1	1
P3	2	3
P4	1	4
P5	5	2

The processes are assumed to have arrived in the order P1, P2, P3, P4, P5, all at time 0.

- a. Draw four Gantt charts illustrating the execution of these processes using FCFS, SJF, a nonpreemptive priority (a smaller priority number implies a higher priority), and RR (quantum = 1) scheduling.

- b. What is the turnaround time of each process for each of the scheduling algorithms in part a?
- c. What is the waiting time of each process for each of the scheduling algorithms in part a?

答案

(a) Gantt 图

FCFS: 0 | P1 | 10 | P2 | 11 | P3 | 13 | P4 | 14 | P5 | 19

SJF: 0 | P2 | 1 | P4 | 2 | P3 | 4 | P5 | 9 | P1 | 19

Priority: 0 | P2 | 1 | P5 | 6 | P1 | 16 | P3 | 18 | P4 | 19

RR(q=1): 0 | P1 | 1 | P2 | 2 | P3 | 3 | P4 | 4 | P5 | 5 | P1 | 6 | P3 | 7 | P5 | 8 | P1 | 9 | P5 | 10 | P1 | 11 | P5 | 12 | P1 | 13 | P5 | 14 | P1 | 15 | P1 | 16 | P1 | 17 | P1 | 18 | P1 | 19

(b) 周转时间

算法	P1	P2	P3	P4	P5
FCFS	10	11	13	14	19
SJF	19	1	4	2	9
Priority	16	1	18	19	6
RR	19	2	7	4	14

(c) 等待时间

算法	P1	P2	P3	P4	P5
FCFS	0	10	11	13	14
SJF	9	0	2	1	4
Priority	6	0	16	18	1
RR	9	1	5	3	9

题目 9 408 优先级加时间片轮转调度

[408]（本题满分 7 分）系统采用优先级（优先级越大表示优先级越高）与时间片轮转调度算法，仅当发生时钟中断时才触发抢占 CPU 操作，时钟中断间隔为 10 ms。进程首次进入就绪队列时，其时间片为 50 ms。若进程因时间片用完而返回就绪队列，其优先级值减 1；若进程被更高优先级进程抢占而返回就绪队列，其优先级值保持不变。当多个进程优先级相同时，先进入就绪队列的进程优先被调度。四个进程的到达时刻、初始优先级与 CPU 运行时间如下表所示：

进程	到达就绪队列时间 (ms)	优先级	CPU 运行时间 (ms)
P1	10	3	95
P2	10	4	20
P3	12	2	40
P4	14	5	60

- (1) 从 10 ms 开始进程调度，直至所有进程调度结束，此时中断次数与 CPU 调度次数分别为多少？P1、P2、P3、P4 各自的首次调度发生在哪个时刻？
- (2) 若时间片由 50 ms 改为 100 ms，CPU 调度次数将增大、不变还是减少？若时钟中断间隔由 10 ms 改为 1 ms，系统开销将增大、不变还是减少？

答案

- (1) 调度过程为：
- 10--20: P2, 20--70: P4, 70--80: P2, 80--90: P4, 90--140: P1, 140--180: P3, 180--225: P1
- 因此：
- 1. 中断次数 = 21
 - 2. CPU 调度次数 = 7
 - 3. 首次调度时刻：P1 在 90 ms，P2 在 10 ms，P3 在 140 ms，P4 在 20 ms
- (2) 时间片改为 100 ms 时，CPU 调度次数减少；时钟中断间隔改为 1 ms 时，系统开销增大。

题目 10 多级反馈队列调度

Given the table below showing the process information of a system, based on multilevel feedback queuing scheduling scheme. Suppose there are five levels in the system and within each level the FCFS scheduling is used. If a process is preempted, it will wait for being scheduled at the first place in the FCFS queue. Draw a Gantt chart to show the time of CPU allocated to each process until all processes are finished using time quantum $q = 2^i$, where i ranges from 0 to 4 indicating the i th level queue.

Process	Arrival Time	Service Time
P1	0	3
P2	1	5
P3	3	2
P4	4	9
P5	5	12

答案

Gantt 图为:

0 | P1 | 1 | P2 | 2 | P1 | 3 | P3 | 4 | P4 | 5 | P5 | 6 | P1 | 7 | P2 | 9 | P3 | 10 | P4 | 12 | P5 | 14 | P2 | 16 | P4 | 20 | P5 | 24 | P4 | 26 | P5 | 31

4 进程同步与死锁

题目 11 忙等待

What is the meaning of the term busy waiting? What other kinds of waiting are there in an operating system? Can busy waiting be avoided altogether? Explain your answer.

答案

Busy waiting refers to a situation where a process, while waiting for a condition to be met, does not block itself but instead continuously loops to check the condition. In operating systems, there are also other forms such as blocking wait, sleep wait, and I/O wait. While busy waiting cannot be entirely avoided, it should only be used in scenarios involving extremely short wait times or low-level kernel synchronization.

题目 12 银行家算法

Consider the following snapshot of a system. Answer the following questions using the banker's algorithm:

	Allocation				Max				Available			
	A	B	C	D	A	B	C	D	A	B	C	D
P0	0	0	1	2	0	0	1	2	1	5	2	0
P1	1	0	0	0	1	7	5	0				
P2	1	3	5	4	2	3	5	6				
P3	0	6	3	2	0	6	5	2				
P4	0	0	1	4	0	6	5	6				

- a. What is the content of the matrix Need?
- b. Is the system in a safe state?
- c. If a request from process P1 arrives for (0, 4, 2, 0), can the request be granted immediately?

答案

(a) Need Matrix

Process	A	B	C	D
P0	0	0	0	0
P1	0	7	5	0
P2	1	0	0	2
P3	0	0	2	0
P4	0	6	4	2

- (b) The system is in a safe state.
- P0 -> P2 -> P3 -> P4 -> P1
- (c) Request (0,4,2,0) can be granted immediately.

题目 13 Dekker 算法

The first known correct software solution to the critical-section problem for two processes was developed by Dekker. The two processes, P0 and P1, share the following variables:

```
boolean flag[2]; /* initially false */    int turn;
```

The structure of process P_i ($i = 0$ or 1) is shown in the following figure. Prove that the algorithm satisfies all three requirements for the critical-section problem.

```
while (true) {
    flag[i] = true;
    while (flag[j]) {
        if (turn == j) {
            flag[i] = false;
            while (turn == j)
                ; /* do nothing */
            flag[i] = true;
        }
    }
    /* critical section */
    turn = j;
    flag[i] = false;
    /* remainder section */
}
```

答案

The Dekker algorithm satisfies all three requirements for the critical-section problem: mutual exclusion, idle let-in, and bounded waiting.

题目 14 植树问题

[408] 三个人一起植树，甲挖坑，乙放树苗入坑并填土，丙负责为新种树苗浇水。步骤依次为：挖树坑，放树苗，填土和浇水。现在有铁锹和水桶各一个，铁锹用于挖树坑、填土，水桶用于浇水。当树坑数量小于 3 时，甲才可以挖树坑。设初始坑 = 0，铁锹、水桶均可用，定义尽可能少的信号量，用 `wait()` 和 `signal()` 操作描述植树过程中三人的同步互斥关系，并说明所用信号量的作用及其初值。

答案

可定义 4 个信号量：

1. `empty = 3`：还能再挖的坑位数
2. `dug = 0`：已挖好、等待放苗填土的坑数
3. `filled = 0`：已填土、等待浇水的树苗数
4. `shovel = 1`：铁锹互斥信号量

进程描述如下：

```
// 甲：挖坑
while (true) {
    wait(empty);
    wait(shovel);
    dig();
    signal(shovel);
    signal(dug);
}

// 乙：放树苗并填土
while (true) {
    wait(dug);
    wait(shovel);
    plant();
    fill();
    signal(shovel);
    signal(empty);
    signal(filled);
}

// 丙：浇水
while (true) {
    wait(filled);
    water();
}
```

题目 15 缓冲区 B 的同步互斥

[408] 计算机系统中的进程之间往往需要相互协作以完成一个任务，在某网络系统中缓冲区 B 用于存放一个数据分组，对 B 的操作有 C1、C2 和 C3。C1 将一个数据分组写入 B 中，C2 从 B 中读出一个数据分组，C3 对 B 中的数据分组进行修改。要求 B 为空时才能执行 C1，B 非空时才能执行 C2 和 C3。请回答下列问题：

- (1) 假设进程 P1 和 P2 均需执行 C1，实现 C1 的代码是否为临界区？为什么？
- (2) 假设 B 初始为空，进程 P1 执行 C1 一次，进程 P2 执行 C2 一次。请定义尽可能少的信号量，并用 wait()、signal() 操作描述进程 P1、P2 之间的同步或互斥关系，说明所用信号量的作用及初值。
- (3) 假设 B 初始不为空，进程 P1 和 P2 各执行 C3 一次。请定义尽可能少的信号量，并用 wait()、signal() 操作描述进程 P1 和 P2 之间的同步或互斥关系，说明所用信号量的作用及初值。

答案

- (1) 是临界区，因为两个进程都要访问同一个缓冲区 B，且都要求 B 为空才能写入，必须互斥。
- (2) 只需 1 个信号量 `full = 0`，表示 B 中是否已有可读分组。
- (3) 只需 1 个信号量 `mutex = 1`，表示对缓冲区 B 的互斥访问。

```
// (2) B 初始为空，P1 执行 C1 一次，P2 执行 C2 一次
semaphore full = 0;
```

```
P1:
    C1;
    signal(full);
```

```
P2:
    wait(full);
    C2;
```

```
// (3) B 初始非空，P1 和 P2 各执行 C3 一次
semaphore mutex = 1;
```

```
P1:
    wait(mutex);
    C3;
    signal(mutex);
```

```
P2:
    wait(mutex);
    C3;
    signal(mutex);
```

5 内存管理

题目 16 页大小选择题

Which statement about the page size is correct?

- A. A smaller page size leads to less TLB miss.
- B. A smaller page size leads to smaller page table.

- C. A smaller page size leads to less I/O overhead.
- D. A smaller page size leads to smaller internal fragmentation.

答案

The correct statement is: **D**

题目 17 页面置换与工作集

Consider the reference page sequence:

1, 2, 3, 4, 2, 3, 1, 4, 5, 6, 7, 4, 5, 6, 7

The number of page frames is 3.

- (a) How many page faults occur for FIFO? Fill in the answer and show how the page faults are generated.
- (b) How many page faults occur for LRU? Fill in the answer and show how the page faults are generated.
- (c) How many page faults occur for OPT? Fill in the answer and show how the page faults are generated.
- (d) Which of the three algorithms achieves the worst page-fault rate?
- (e) Use this reference string to describe what thrashing is.
- (f) With this reference string, how many frames are required at the timing of each reference? Use the working set model with window size $\Delta = 5$ to provide your reason.

答案

- (a) Number of FIFO page faults: **12**
- (b) Number of LRU page faults: **13**
- (c) Number of OPT page faults: **9**
- (d) The worst performer: **LRU**
- (e) The working set increases significantly in the latter half of the string, while there are only 3 physical blocks; this leads to frequent page swapping and exhibits thrashing.
- (f) Under the working set model with $\Delta = 5$, the number of page frames required for each access is:

1, 2, 3, 4, 4, 3, 4, 4, 5, 5, 5, 4, 4, 4, 4

FIFO:

1(F)→[1]
2(F)→[1,2]
3(F)→[1,2,3]
4(F)→[4,2,3]
2(H)→[4,2,3]
3(H)→[4,2,3]
1(F)→[4,1,3]
4(H)→[4,1,3]
5(F)→[4,1,5]
6(F)→[6,1,5]
7(F)→[6,7,5]
4(F)→[6,7,4]
5(F)→[5,7,4]
6(F)→[5,6,4]
7(F)→[5,6,7]

LRU:

1(F)→[1]
2(F)→[1,2]
3(F)→[1,2,3]
4(F)→[4,2,3]
2(H)→[4,2,3]
3(H)→[4,2,3]
1(F)→[1,2,3]
4(F)→[1,4,3]
5(F)→[1,4,5]
6(F)→[6,4,5]
7(F)→[6,7,5]
4(F)→[6,7,4]
5(F)→[5,7,4]
6(F)→[5,6,4]
7(F)→[5,6,7]

OPT:

1(F)->[1]

2(F)->[1,2]

3(F)->[1,2,3]

4(F)->[4,2,3]

2(H)->[4,2,3]

3(H)->[4,2,3]

1(F)->[4,1,3]

4(H)->[4,1,3]

5(F)->[4,5,3]

6(F)->[4,5,6]

7(F)->[4,5,7]

4(H)->[4,5,7]

5(H)->[4,5,7]

6(F)->[6,5,7]

7(H)->[6,5,7]

题目 18 页表地址转换

Consider the page table for a system with 12-bit virtual and physical addresses and 256-byte pages. The list of free page frames is D, E, F (that is, D is at the head of the list, E is second, and F is last). A dash for a page frame indicates that the page is not in memory. Convert the following virtual addresses to their equivalent physical addresses in hexadecimal. All numbers are given in hexadecimal.

Page	Page Frame
0	-
1	2
2	C
3	A
4	-
5	4
6	3
7	-
8	B
9	0
9EF	111 700 0FF

答案

Virtual Address		Result	Physical Address
9EF		Hit	0EF
111		Hit	211
700	Miss, allocate page frame D		D00
0FF	Miss, allocate page frame E		EFF

题目 19 进程虚拟地址空间分析

[408] 某进程的虚拟地址空间如图，阴影部分为未占用区域，有 C 程序：

操作系统内核区
用户栈
共享库的存储映射区
动态生成的堆（运行时由 malloc 创建）
读/写数据段
只读代码段
未使用区

```
char *ptr;

void main() {
    int length;
    ptr = (char*) malloc(100);
    scanf("%s", ptr);
    length = strlen(ptr);
    printf("length=%d\n", length);
    free(ptr);
}
```

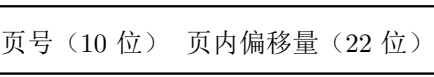
- (1) 上述程序执行时，PCB 位于哪个区域，执行 `scanf()` 等待键盘输入时，该进程处于什么状态？
- (2) `main()` 函数的代码位于哪个区域？其直接调用的哪些函数的功能需要通过执行驱动程序实现？
- (3) 变量 `ptr` 被分配在哪个区域？若变量 `length` 没有被分配在寄存器中，则会被分配在哪个区域？`ptr` 指向的字符串位于哪个区域？

答案

- (1) PCB 位于操作系统内核区；`scanf()` 等待键盘输入时，进程处于阻塞态。
- (2) `main()` 的代码位于只读代码段；直接调用中需要通过驱动程序实现功能的是 `scanf()` 和 `printf()`。
- (3) `ptr` 位于读/写数据段；`length` 若未分配到寄存器，则位于用户栈；`ptr` 指向的字符串位于堆区。

题目 20 32 位页式虚拟存储管理

[408] 某计算机按字节编址，采用页式虚拟存储管理方式，虚拟地址和物理地址的长度均为 32 位，页表项大小为 4 字节，页大小为 4 MB，虚拟地址结构如下：



进程 P 的页表起始虚拟地址为 B8C0 0000H，被装载到从物理地址 6540 0000H 开始的连续主存空间中。请回答下列问题：

- (1) 若 CPU 在执行进程 P 的过程中，访问虚拟地址 1234 5678H 时发生了缺页异常，经过缺页异常处理和 MMU 地址转换后得到的物理地址是 BAB4 5678H。在此次缺页异常处理中，需要为新缺页分配页框并更新相应页表项，则该页表项的虚拟地址和物理地址分别是什么？该页表项中的页框号更新后的值是什么？
- (2) 进程 P 的页表所在页的页号是什么？该页对应页表项的虚拟地址是什么？该页表项中的页框号是多少？

答案

- (1) 对应页表项的虚拟地址为 B8C0 0120H，物理地址为 6540 0120H，更新后的页框号为 2EAH。
- (2) 页表所在页的页号为 2E3H，对应页表项的虚拟地址为 B8C0 0B8CH，该页表项中的页框号为 195H。

6 文件系统

题目 21 408 文件系统综合题

[408]（本题满分 8 分）文件系统的目录项包括文件名和索引节点号。磁盘包含索引节点表、位图、目录、文件数据等元数据。若盘块大小为 4 KB，盘块号占 4 B，索引节点表存放了系统的所有文件，从 0 开始编号，存放在盘块号 100 开始连续的 4096 个盘块中。索引节点占用 128 B，包含直接地址项 5 个，一级间接地址项、二级间接地址项、三级间接地址项各 1 个。磁盘位示图和索引节点位示图分别记录磁盘和索引节点的使用情况，0 表示未使用，1 表示已使用。其中目录结构图与文件的索引节点表如下所示，file 文件占 30 KB。



- (1) file 的索引节点所在的盘块号是多少？若 file 的索引节点已经读取到内存，要访问 file 文件中偏移地址 21460 的一个字节数据，则最多需要读多少个盘块？如果文件系统中有足够的磁盘空间，则最多可以存放多少个文件？
- (2) 如果要删除目录 dir1，则需要对元数据进行哪些操作？

答案

- (1)

1. file 的索引节点所在盘块号是 131

2. 访问偏移地址 21460 的一个字节，最多需要再读 2 个盘块

3. 最多可以存放 131072 个文件
- (2)

删除 dir1 时，需要更新目录项、索引节点表、索引节点位图和磁盘位图；若 dir1 中包含 file，还要先删除 file 的相关元数据，再删除 dir1 自身的元数据。